AI Engineering and Claude Architect · Part V, Claude Code

Claude Code, Lesson 1

How Claude Code Works and Its Built-In Tools

Reading a whole project to answer one question cost about 30,000 tokens. A search cost about 500. Claude Code is an agent loop with a fixed budget of attention, and choosing tools well is how you spend it.

Module	AI12 of 13
Lesson	1 of 5
Prerequisites	AI03 L6, AI06 L2 and L7, AI09 L3
Exam relevance	**Core: TS 2.5.** Also TS 1.1, 5.4
Reading time	~30 minutes

Contents

1. How Claude Code Works

Lesson 1 of 5 · Claude Code

How Claude Code Works

Key idea

Claude Code is the agent loop from AI06, plus tools, permissions and context management, so its limits are context limits. Search before you read: Grep for contents, Glob for paths. Edit needs text that appears exactly once, and Read plus Write is the fallback. Explore incrementally, rather than loading everything up front.

> *Before: the agentic loop, context engineering, and sandboxes. Teaches: Claude Code's loop and surfaces, the built-in tools and how to choose them, Edit's rules, incremental exploration, and permission modes. Used by: the rest of AI12, and AI13.*

The hook: one question, three costs

A developer asks: "Where is `RefundPolicy` used?" I measured three ways to answer it on this course's own AI08 project. **Tokens here are estimated as characters divided by 4, which is rough.** Real output from `ExploreCost.java`:

```
AI08 project: 37 Java files, about 30,440 tokens
Question: where is RefundPolicy used?
  read every file:          about 30,440 tokens
  grep only (22 lines):      about    490 tokens
  grep, then read 5 files:   about  7,320 tokens
```

The search is about 60 times cheaper than reading everything, and the combination still costs a quarter. It matters because everything that's read stays in the conversation and is re-sent on each later turn (AI03 L6). **In a long session, a careless first read taxes every turn after it.**

Intuition: a new colleague and a large archive

A new colleague asks where the refund logic lives. You wouldn't tell them to read the whole archive first. You'd say: search the index for "refund," open the two files it finds, and follow the references from there. They'd then have room in their head for the actual task.

Mechanism

What it is

Claude Code runs the loop you built in AI06 L2. The model asks for a tool, the harness runs it, and the result goes back into the conversation, until the model finishes. The docs describe the Agent SDK as giving you the same tools, agent loop and context management that power Claude Code.

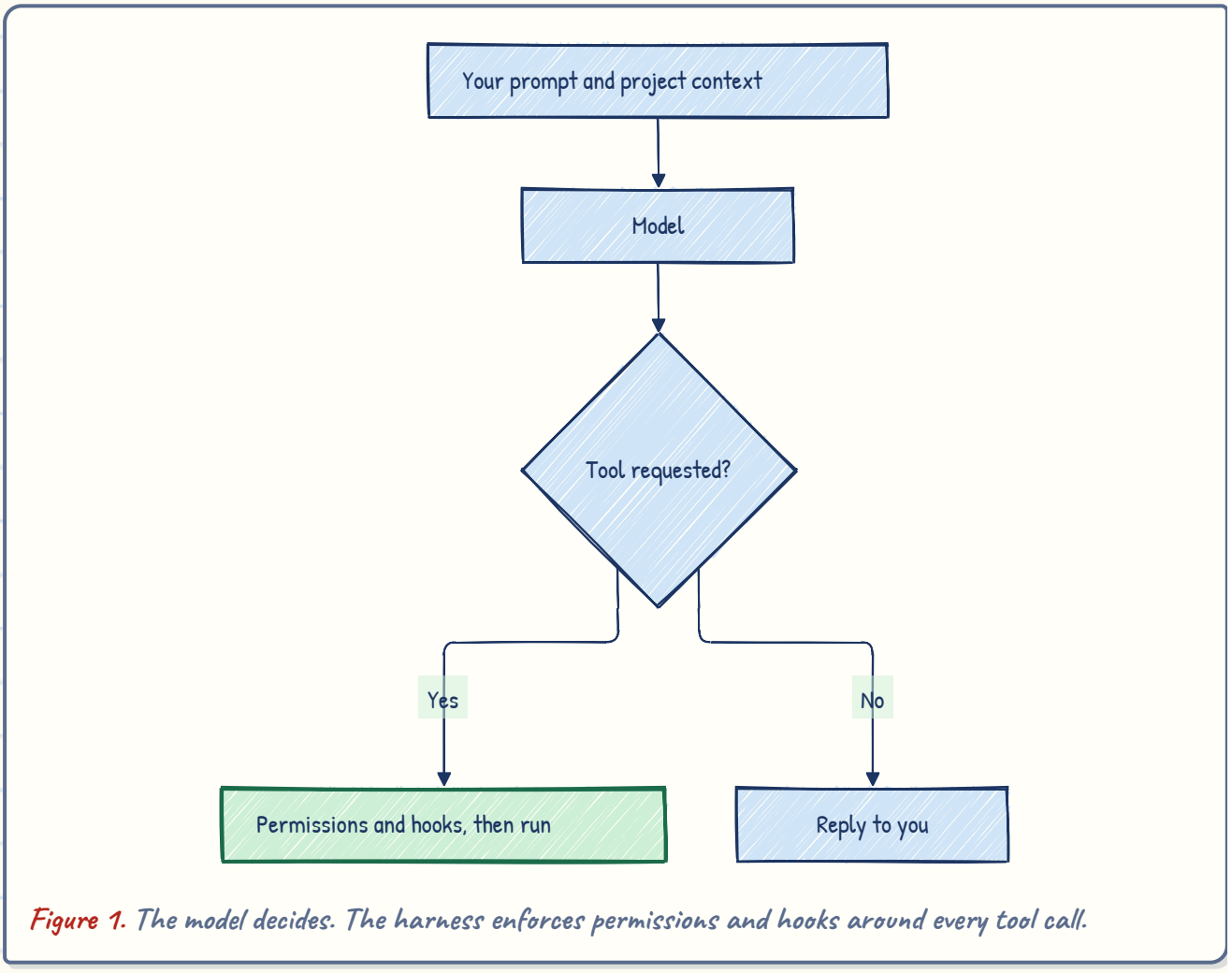


Figure 1. The model decides. The harness enforces permissions and hooks around every tool call.

Its best practices page makes a useful split, which runs through this module:

Kind	Who acts	Examples
Non-deterministic	The model reads it and decides	Your prompt, CLAUDE.md, skills
Deterministic	The harness has to apply it	Hooks, deny rules, managed settings

Surfaces. The CLI, IDE extensions, the desktop app and the web are all ways of reaching it. The exam concentrates on the CLI and the Agent SDK.

The built-in tools

From the tools reference. The exam guide names the first six.

Tool	Does	Permission needed?
Read	Reads a file	No
Write	Creates or overwrites a file	Yes
Edit	Exact string replacement	Yes
Bash	Runs shell commands	Yes
Grep	Searches file contents (built on ripgrep)	No
Glob	Finds files by name pattern	No
Agent	Spawns a subagent (L3)	No
Skill	Runs a skill (L3)	Yes
WebFetch, WebSearch	Reach the web	Yes

Exam answer and what to build today

The guide teaches **Grep** and **Glob** as tools, and that's the exam's answer. The current docs say that on macOS, Linux and WSL those two are out of the default tool set, and Claude searches with `find` and `grep` through Bash, unless you name them in `--tools` or `--allowedTools`. On Windows they're in the default set. The principle is the same either way: search content, match paths, and don't read everything.

Grep or Glob?

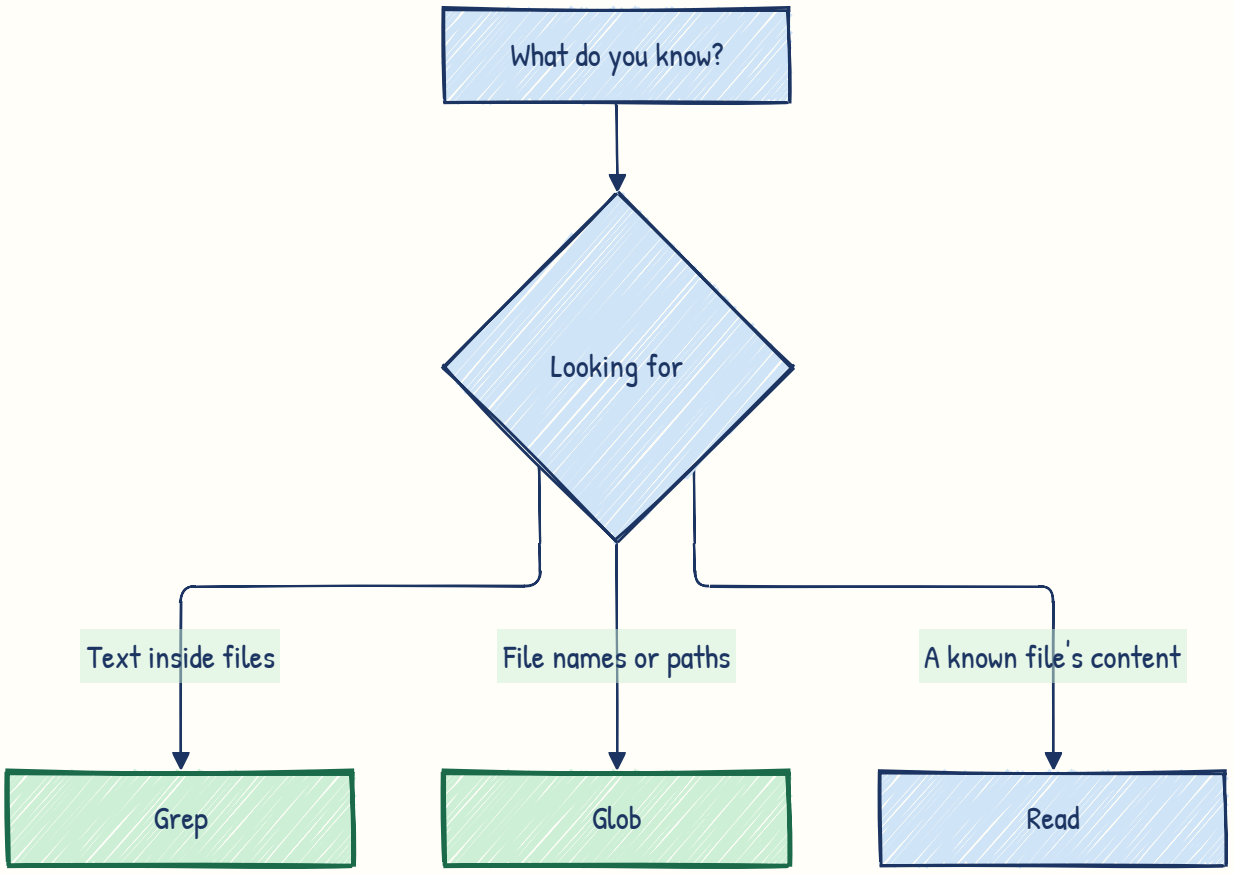


Figure 2. The question decides the tool. Contents go to Grep, and names to Glob.

Question	Tool	Pattern
Who calls `processRefund`?	Grep	`processRefund\(`
Where's the error "quota exceeded" raised?	Grep	`quota exceeded`
Which files import `PaymentClient`?	Grep	`import .*PaymentClient`
Where are the React tests?	Glob	`**/*.test.tsx`
Which migrations exist?	Glob	`db/migrations/*.sql`

Glob needs no file contents at all, so it's very cheap:

```
Question: which files are tests? A path match needs no file contents:
  10 of 37 files, about 545 tokens of paths
```

Grep's pattern syntax is ripgrep's, so regex metacharacters need escaping: finding `interface{}` takes `interface\{\}`. A rejected pattern now returns an error. The docs note that before v2.1.208 it was reported as `No files found`, which reads like a real answer.

Exploring incrementally

The guide's recommended sequence:

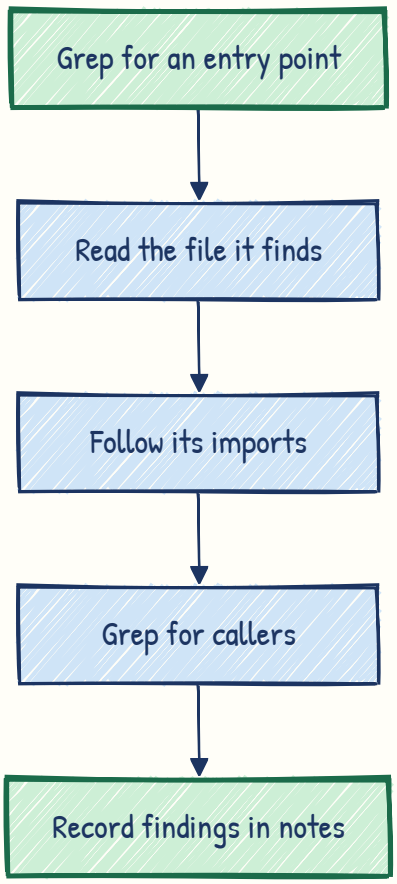


Figure 3. Each step narrows the next. The context holds what's relevant, and not the whole repository.

Tracing through wrappers. A function may be re-exported under other names, so searching for its original name misses callers. First find all exported names, then search for each one. That's a guide skill, and it's a place where a "complete" grep result can quietly be wrong.

Delegate verbose searches. A broad search that returns thousands of lines belongs in a subagent, which returns a summary (TS 5.4, and L4 for Explore).

Edit, and when it fails

Edit does exact string replacement, without regex or fuzzy matching. From the tools reference, it must pass these checks:

Check	Meaning
Read first	Claude must have read the file in this conversation, with some exceptions on newer models
Match	`old_string` appears exactly as written, including whitespace
Uniqueness	`old_string` appears exactly once

Uniqueness is what trips people up. Here's a real file:

```
Edit needs its old_string to appear exactly once in AgentService:
  5 match(es) FAILS  ledger.finish(
  1 match(es) OK     ledger.finish(call.id(), "OK", output);
  6 match(es) FAILS  return new Result(
  1 match(es) OK     case "refusal" -> {\n return new Result(
```

There are three ways out:

Option	When	Source
A longer string with surrounding context	You want one specific occurrence	Tools reference
`replace_all: true`	You want all of them changed	Tools reference
Read, then Write the whole file	Edit can't find a unique anchor	**The exam guide's named fallback**

Exam answer and what to build today

If a question asks for the fallback when Edit fails on non-unique text, the guide's answer is **Read + Write**. In practice, a longer anchor is usually smaller and safer, as writing a whole file risks overwriting other changes.

Permissions in one view

AI09 L3 covered this as security. Here's how a session runs. From the permission modes page:

Mode	What runs without asking
Manual (`default`)	Reads only
`acceptEdits`	Edits, plus common filesystem commands in the working directory
Plan (`plan`)	Reads, with no source edits (L4)
Auto (`auto`)	Everything, with a classifier reviewing actions
`dontAsk`	Anything that would prompt is denied. For CI and scripts
`bypassPermissions`	Everything. For isolated containers and VMs only

Deny rules block in every mode, including `bypassPermissions`. Switch modes with `Shift+Tab`, or start with `--permission-mode`. Per the docs, new interactive sessions default to auto mode on v2.1.283 or later, while `claude -p` starts in Manual.

Why the obvious fixes fall short

Obvious idea	Why it falls short
Read everything first, for full understanding	About 60× the cost of a search here, and it stays in context
Use Glob for "who calls X?"	Glob sees names, and not contents
Use Grep to find files by extension	Works, though it's the wrong tool and returns more
Retry Edit until it matches	Text doesn't become unique by retrying
Grep only the original function name	Misses re-exported aliases
Bigger context window	Doesn't fix attention (AI03 L6)

How Anthropic solved it

Mechanism	What's published	Source
Tool loop	Same loop in CLI and SDK	Agent SDK overview
Tool list	Names, behavior, permissions	Tools reference
Edit's checks	Read-first, match, uniqueness	Tools reference
Modes	Six permission modes	Permission modes page
Compaction	Automatic before the window fills	Best practices
Explore	Read-only subagent	Subagents page

The production architecture

This is a recommended reference design, not a specific company's system.

This is about how a team sets up an exploration-heavy agent, using the SDK or CLI.

Baseline: the agent has all tools, and reads what it likes.

Improved: search first, with a clear brief. New problem: long investigations still fill the context with search output.

Production: split reading from coordination.

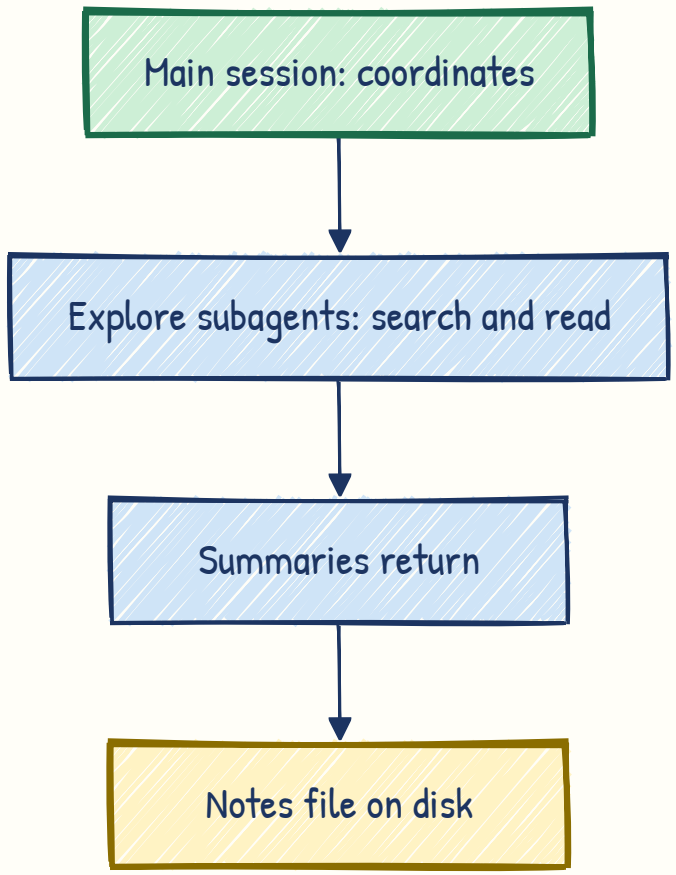


Figure 4. Verbose reading happens in isolated subagents. Findings live in a file, which the main session rereads, and which outlasts any context window.

Component	Why it exists	How it communicates
Main session	Holds the goal and decisions	Agent tool calls
Explore subagents	Keep search output out of the main context	Prompt in, summary out
Notes file	Survives compaction and crashes (TS 5.4)	Read and Edit
Permission mode	Reads free, and writes controlled	Settings
CLAUDE.md	Commands and gotchas (L2)	Loaded at start

Failures:

Failure	How you'd notice	What to do
Edit fails on repeated text	Tool error	Longer anchor, `replace_all`, or Read + Write
Edit misses by whitespace	Tool error	Re-read, and copy the exact text
Grep misses callers	Bug in an alias path	Enumerate exports first
Bad regex	Error, or nothing on old versions	Escape metacharacters
Context fills early	Vague answers (TS 5.4)	Subagents, `/compact`, narrower search
A big Write overwrites other work	Lost changes in the diff	Prefer Edit, and review diffs

Java

The search is simple, and its output is what enters the context: one line per match, and not the file.

```
hits.add(file.getFileName() + ":" + n + ": " + line.strip());
```

That's why a grep result costs so little. The uniqueness check is a count, which is worth running in your head before asking for an Edit:

```
at = text.indexOf(target, at + 1);
```

I measured file sizes and counts, and ran no model. Real sessions also spend tokens on system prompts, reminders and thinking.

Trade-offs

Decision	Benefit	Cost
Grep first	Very cheap	Can miss aliases, and needs a good term
Read whole files	Full local context	Stays in the history
Edit	Small, reviewable change	Needs a unique anchor
Write	Always applies	Overwrites everything
Subagent search	Clean main context	Delegation cost, and a summary can lose details

When not to be frugal: a small repository with a focused task can just be read. Don't spend effort optimizing a 200-line project.

Common mistakes

Confusing Grep and Glob. "Find all files that call X" sounds like finding files, and it's a content search.

Common mistakes

Treating a grep's empty result as proof. Check your pattern, aliases and generated code before concluding something is unused.

What to say in an interview

"Claude Code is an agent loop, and context is the budget. I search first: Grep for contents, Glob for paths, then Read what matters and follow imports. Verbose searches go to subagents. Edit needs a unique match, so I use a longer anchor or fall back to Read plus Write. And I remember that everything read stays in the conversation for every later turn."

Certification lens

How this lesson maps to CCAR-F

TS 2.5 is exam content: Grep for content, Glob for paths, Read/Write for full files, Edit for targeted changes, Read + Write as the fallback, incremental exploration, and tracing through wrappers. It supports TS 1.1 and TS 5.4. Permission mode details aren't in the guide's objectives.

These are original practice questions, not real or leaked.

Q Q1 · Exam style, scenario: Developer Productivity · Easy. You need every place that throws `InvalidTokenException`. Which tool fits?

A. Glob with `**/*Exception*`

B. Write, to create an index file first

C. Grep for `throw new InvalidTokenException`

D. Read on each file in `src/`

A C (TS 2.5). It's a content search. A matches names only, B does nothing useful, and D wastes context.

Q Q2 · Exam style, scenario: Developer Productivity · Medium. A config key appears 14 times in one file, and you need to change one occurrence. Edit keeps failing. What's the guide's reliable approach?

A. Read the file, then Write the modified version

B. Retry Edit with the same string

C. Use Glob to select the occurrence

D. Delete the other 13 occurrences first

A A (TS 2.5). It's the named fallback for a non-unique match. B can't succeed, Glob works on paths, and D changes behavior. In practice, a longer anchor also works.

Q Q3 · Exam style, scenario: Code Generation · Medium. You must understand how orders are priced in an unfamiliar service. What's the best first move?

A. Read every file under `src/` so nothing's missed

B. Ask Claude how pricing usually works in such services

C. Rewrite the module, so that it's easier to understand

D. Grep for pricing entry points, then Read and follow the imports

A D (TS 2.5). Incremental exploration keeps context useful. A exhausts it, B is general knowledge and not this code, and C is premature.

Q Q4 · Exam style, scenario: Developer Productivity · Hard. `fetchUser` is exported as `getUser` and `loadUser` by wrapper modules, and a grep for `fetchUser` finds few callers. What's next?

A. Conclude that it's rarely used

B. Identify all of the exported names, then Grep for each

C. Glob for files whose names include "user"

D. Read the wrapper modules, as they hold all the callers

A **B** (TS 2.5). Aliases hide callers. A is a false conclusion, C works on file names, and wrappers re-export rather than containing the callers.

Q Q5 · Why does the cost of a file read go on after the turn it happened in?

A The API is stateless, so the conversation, including that tool result, is sent on each later request (AI03). Prompt caching reduces the cost, and the content still occupies the context window until compaction or a clear.

Revision - Lesson 1

→ Claude Code is the agent loop with tools, permissions and context management.

→ Prompts and CLAUDE.md are non-deterministic. Hooks and deny rules are deterministic.

→ Grep searches contents, and Glob matches paths. Search before reading.

→ On Linux and macOS, current defaults search through Bash. The guide's Grep and Glob remain the exam's answer.

→ Explore incrementally: entry point, read, follow imports, find callers.

→ Trace aliases by listing all exported names, and search for each.

→ Edit needs read-first, an exact match, and uniqueness.

→ Guide's fallback is Read + Write. A longer anchor or `replace_all` also work.

→ Permission modes: Manual, `acceptEdits`, plan, auto, `dontAsk`, `bypassPermissions`. Deny rules always apply.

Your notes